Ders 5 Genişletilmiş Notu: Decorator, Facade, Flyweight, Proxy ve Chain of Responsibility

Ders 5 Genişletilmiş Notu: Decorator, Facade, Flyweight, Proxy ve Chain of Responsibility

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Decorator Kalıbı: Amaç ve Motivasyon
- 2. Decorator'ın Yapısı ve Kullanım Durumları
- 3. Decorator Uygulama Örneği: Satış Makbuzu
- 4. Decorator'da Zincirleme Sırası
- 5. Facade Kalıbı
- 6. Flyweight Kalıbı: Amaç ve Metin Editörü Örneği
- 7. Flyweight'in Yapısı ve Kullanım Durumları
- 8. Flyweight Uygulama Örneği: Kimya Fabrikası
- 9. Proxy Kalıbı
- 10. Proxy Uygulama Örneği: Pay-per-view Yayın Sunucusu
- 11. Davranışsal Kalıplara Giriş
- 12. Chain of Responsibility Kalıbı
- 13. Chain of Responsibility Uygulama Örneği: Reklam Yönetim Yazılımı
- Örnek Sorular

1. Decorator Kalıbı: Amaç ve Motivasyon

Önceki derste yapısal (structural) kalıplardan Adapter, Bridge ve Composite işlenmişti. Bu ders, yapısal kalıplardan Decorator ile başladı; ardından Facade, Flyweight ve Proxy işlendi, dersin sonunda da davranışsal (behavioral) kalıplara geçilerek Chain of Responsibility anlatıldı. Hoca tasarım kalıplarının kendi kendine çalışmaya (self-study) uygun bir konu olduğunu da hatırlattı.

1.1 Amaç

Decorator kalıbının amacı, nesnelere çalışma anında, yani dinamik olarak, ek sorumluluklar atamaktır.

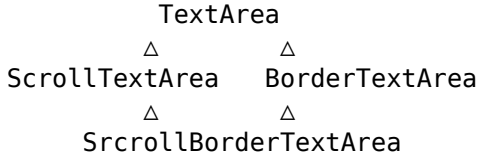
Nesnelere ek sorumluluk normalde kalıtımla verilir: mevcut sınıftan türetilen alt sınıf, üst sınıfın bütün özelliklerine ve ek olarak kendisine kodlanan özelliklere sahip olur. Ama kalıtım kodlama (derleme, compile time) anında belirlenen, çalışma anında değiştirilemeyen bir ilişkidir. Üstelik sınıf düzeyinde bir ilişki olduğu için aynı alt sınıftan oluşturulan bütün nesneler aynı niteliklere sahip olur. Sorumlulukları nesnelere çalışma anında atayabilirsek, tek tek nesneleri (bireysel nesne düzeyinde) farklı biçimlerde özelleştirebiliriz. Decorator bunu sağlayan kalıplardan biridir.

1.2 GoF örneği: metin alanına kaydırma çubuğu ve çerçeve

Dörtlü Çete'nin (GoF) örneği yine bir GUI çerçevesinden gelir. Tek satırlık bir metin kutusu değil, çok satırlı bir metin alanı (text area) vardır. Bu alana bazen yatay kaydırma çubuğu (scrollbar), bazen dikey kaydırma çubuğu, bazen ikisi birden, bazen de bir çerçeve (border) eklemek isteriz. Üstelik her metin alanı aynı olmayacaktır: kimi çerçeveli ve kaydırmalı, kimi çerçevesiz ama kaydırmalı, kimi yalnız dikey kaydırmalı olacaktır.

Çözüm 1, kalıtımla. `TextArea` sınıfından `ScrollTextArea` (yatay/dikey ayrımı şimdilik yapılmıyor), `BorderTextArea` ve ikisini birden isteyen durum için `ScrollBorderTextArea` kardeş alt sınıfları türetilir. Kaydırma için bir iş mantığı, çerçeve için başka bir iş mantığı yazılır; `ScrollBorderTextArea`'da bu iki mantık tekrar edilmek zorundadır, yani kopyala-yapıştır kod oluşur. Kaydırma yatay ve dikey diye ayrılırsa sınıf sayısı daha da artar.

Çözüm 2, çoklu kalıtımla (ekrandan, 00:10:00). `ScrollBorderTextArea`, kardeş sınıf yapılmak yerine hem `ScrollTextArea`'nın hem `BorderTextArea`'nın alt sınıfı yapılırsa kaydırma ve çerçeve mantığı yeniden kodlanmaz. Buna çoklu kalıtım (multiple inheritance) denir.



Slayttaki çözümün zayıf yönleri şunlardır:

- Çoklu kalıtım her dilde desteklenmez.
- Çoklu kalıtımın bu çözümdeki şekli "diamond problem" sorununun bir örneğidir.
- `TextArea`'ya ek olarak bir de `Panel` sınıfına kaydırma ve çerçeve yeteneği kazandırmak için ek üç sınıf daha gerekecektir. Üstelik bu durumda kaydırma ve çerçeve yeteneklerinin tek yerde gerçeklenmesi avantajı da ortadan kalkacaktır.

  Ders sonrası not: Slayttaki sınıf adı `SrcrollBorderTextArea` biçiminde yazılmıştır (ekrandan, 00:10:00); doğrusu `ScrollBorderTextArea`'dır.

1.3 Diamond problem

Hoca, slaytta sınıfları ve kalıtım ilişkilerini işaretleyince ortaya çıkan şeklin bir elmasa benzediğini, bu yüzden soruna "diamond problem" (elmas kalıtım sorunu) dendiğini gösterdi. Sorun şudur: `TextArea` sınıfında soyut olarak tanımlanmış bir metot, `ScrollTextArea` ve `BorderTextArea`'da farklı gövdelerle somut olarak tanımlanmış olsun. Bu metot `ScrollBorderTextArea`'da yeniden tanımlanmazsa, kalıtımla hangi ebeveynin metodu aşağı aktarılacaktır? Bu bir belirsizliktir.

Diller bu belirsizliği farklı biçimlerde çözebilir: önce hangi ebeveynden kalıtım alınmışsa onunki geçerli olur; ya da tersine, en son hangi ebeveynden kalıtım alınmışsa onunki geçerli olur; ya da C++'ta olduğu gibi bu tür metotlar pure virtual tanımlanır ve her hâlükârda en alttaki sınıfın onu tanımlaması gerekir. Fakat bu çözümlerin getirdiği ek karmaşıklık sağladığı yarardan fazla görüldüğü için güncel nesneye dayalı dillerde çoklu kalıtım desteklenmez. Java'da ve C#'ta çoklu kalıtım yoktur, C++ ise destekler. Hoca bunun yerine kullanılabilecek çok daha iyi çözümler olduğunu, Decorator'ın da bunlardan biri olduğunu söyledi.

1.4 Decorator'ın önerdiği çözüm

Önerilen çözümde sınıflar değil nesneler birbiriyle ilişkilendirilir ve her sorumluluk ayrı bir nesnede olur: çerçeve çizmek bir nesnede, kaydırma mantığı başka bir nesnede. Bir `TextView` nesnesi bir kaydırma bileşeniyle (`ScrollDecorator`) ilişkilendirilirse artık kaydırma yeteneğine sahip olur; ikisi birlikte tek bir varlık gibi ele alınır. Ardından mevcut nesne bir `BorderDecorator` nesnesiyle ilişkilendirilirse çerçeve yeteneği de kazanır. Başka bir `TextView` yalnız `BorderDecorator` ile, bir başkası yalnız `ScrollDecorator` ile ilişkilendirilebilir. Böylece nesne düzeyinde farklılaşma ve nesne düzeyinde yeni yetenek ekleme yapılmış olur.

Temel nesne `TextView`'dır. Ona yeni yetenekler ekleyen sınıflara dekoratör (decorator) sınıfları denir ve bunların bir soyut `Decorator` üst sınıfı vardır. Hoca bunun bir arayüz (interface) olup

olamayacağını sordu ve cevabı verdi: olamaz, çünkü ilişkili olduğu nesnenin işinin yaptırılması gerekir. Yani `Decorator` kesinlikle arayüz olamaz, soyut bir sınıf olacaktır. Ek sorumluluk yerine getirildikten sonra, zincirin en ucundaki asıl işi yapacak nesnenin kendi işini yapmasına da izin verilmelidir. Bu iş, örneğin ekrana çizilmek (render edilmek) ise, `TextView`'daki `Draw` metoduyla aynı imzaya sahip somut bir metot soyut `Decorator` sınıfına eklenir ve bu metodun yaptığı asıl şey ilişkili nesnenin bu işi yapmasını sağlamaktır.

Bir `TextView`'ı farklı dekoratörlerle ilişkilendirebilmek için Composite'tekine benzer bir sahiplik ilişkisi kurulur. Hoca slaytta elmasla gösterilmesine rağmen bunun bir dizi değil, birebir sahiplik ilişkisi olduğunu vurguladı: her dekoratörün ilişkili olduğu tek bir bileşen (component) nesnesi vardır. Bu bileşenin hem başka bir dekoratör hem de asıl işi yapan nesne olabilmesi için, `Decorator` ve `TextView`'ın üstüne bir ortak üst sınıf daha konur: `VisualComponent`. Zaten kaydırma çubuğuna ihtiyaç duyan tek bileşen metin alanı olmayabilir; Java'daki paneller ya da bir pencere çerçevesi de işletim sisteminin sağlamadığı, bizim yapmamız gereken kaydırma çubuklarına ihtiyaç duyabilir. Bu tasarımda dekoratör türleri başka görsel bileşenler tarafından da kullanılabilir hâle gelir.

Somut dekoratörler (`ScrollDecorator`, `BorderDecorator`) farklı yetenekleri kodlar ve kendi iş mantıklarıyla ilgili metotlara sahiptir; örneğin en son hangi satırda kalındığını tutan bir kaydırma konumu alanı ya da belirli bir satıra kaydırma metodu. Üst sınıftan alınan `Draw` metodu da burada yeniden tanımlanır ve tanımlanma biçimi önemlidir. `BorderDecorator.Draw()` önce üst sınıfın `Draw` metodunu çağırır; bu çağrı silsile yoluyla en alttaki ilişkili bileşenin `Draw` metoduna kadar gider. Ardından dekoratör kendi ek işini yapar, yani çerçeveyi çizer.

Somut dekoratörde ek yeteneğin yürütülmesi için şu varyasyonlar vardır:

- Önce ek yeteneği yürütmek, sonra akışı ilişkili nesneye iletmek.
- Önce ilişkili nesnenin ve zincirdeki diğer nesnelerin işlerini yapmasına izin vermek, sonra dekoratörün kendi işini yapmak.
- İkisi birden: önce bir şeyler yapmak, sonra işi silsile yoluyla yürütmek, sonra ek olarak bir şeyler daha yapmak.

2. Decorator'ın Yapısı ve Kullanım Durumları

2.1 Kalıp yapısı

Slayttaki kalıp yapısı (ekrandan, 00:20:00):

```
Component {Operation()}  <──────────────── component ──┐
   △── ConcreteComponent {Operation()}                  │
   △── Decorator {Operation()}  ◇──────────────────────┘
          Operation():  component->Operation()
          △── ConcreteDecoratorA {Operation(); addedState}
          △── ConcreteDecoratorB {Operation(), AddedBehavior()}
                 Operation():  Decorator::Operation(); AddedBehavior();
```

- **ConcreteComponent:** Asıl işi yapacak ya da farklı biçimlerde özelleşecek nesnenin sınıfıdır.
- **Component:** Soyut bileşen sınıfıdır; arayüz de olabilir. Asıl yapılan çekirdek işin imzası burada tanımlanır.
- **Decorator:** Kalıba adını veren soyut sınıftır. Mutlaka soyut sınıf olmalıdır, çünkü nesneler arasındaki birebir sahiplik ilişkisini tutmak için bir `component` alanı tanımlanmalı ve çekirdek iş metodunun gövdesinde işi yapacak nesneye yönelik çağrı kodlanmalıdır.
- **ConcreteDecorator:** Farklı ek davranışlar (`AddedBehavior`), ek durum bilgisi (üye alan, `addedState`) ya da ikisini birden ekler. Çekirdek işleme müdahale eder: çekirdek iş yapılmadan önce, sonra ya da hem önce hem sonra gerekli özelleştirmeleri yapar ve çekirdek işin yapılmasına da izin verir.

2.2 Ne zaman kullanılır?

- Tek tek nesnelere dinamik olarak, yani çalışma anında, yeni sorumluluklar eklemek istendiğinde.

- Tek tek nesnelere, aynı türden diğer nesneleri etkilemeden, şeffaf biçimde yeni sorumluluklar eklemek istendiğinde.
- Bazı sorumlulukları iptal etmek ya da askıya almak istendiğinde.
- Kalıtıma alternatif bir tasarım düşünüldüğünde.

Askıya alma konusunda hoca şunu açıkladı: dekoratör çekirdek işlemin öncesinde, sonrasında ya da her iki tarafında bir şeyler yapabildiği gibi, çekirdek işlemin çağrılmasına hiç izin vermeyebilir. Bunu zincirin kırılması olarak düşünmemek gerekir; belki çekirdek sorumluluğun belli bir süre askıya alınması gerekiyordur. Hocanın örneği: bir doktor kendisi hastayken ameliyat edemez. Bu durum modellenebilir: doktor iyileşmişse operasyonu yap, aksi hâlde yapma.

2.3 Gerçek kullanım: Java G/Ç kütüphanesi

Decorator'ın pratikte kullanıldığı bir örnek Java'nın akış (stream) tabanlı G/Ç kütüphanesidir. Java'da `InputStream` ve `OutputStream` sınıfları vardır. Metin modunda işlem yapılacaksa bir `FileInputStream` bir okuyucu sınıfla zincirlenir; aslında arada bir dekoratör ilişkisi kurulmuş olur. Komple nesneler diske kaydedilecek ya da byte'lara çevrilip ağ üzerinden aktarılacaksa `FileOutputStream` bir `ObjectOutputStream` ile zincirlenir, yani `FileOutputStream` `ObjectOutputStream` ile süslenmiş (dekore edilmiş) olur.

> **Ek bilgi (derste anlatılmadı):** Java'da bu zincirleme büyük olasılıkla en sık `new ObjectOutputStream(new FileOutputStream("dosya"))` biçiminde yazılır. Metin okuma tarafında da `new BufferedReader(new InputStreamReader(new FileInputStream("dosya")))` gibi iç içe kurucular kullanılır; her dış sınıf içteki akışa ek bir yetenek (tamponlama, karakter çözme, nesne serileştirme) kazandırır.

2.4 Zayıf yönler

Kalıp yukarıda sayılan biçimlerde kullanılır; başka bir kullanım tarzı yoktur, ama akla gelen ihtiyaçların hemen hepsini karşılar. Karşılayamadığı bir durum çıkarsa başka bir çözüm düşünülür. Dikkat edilmesi gereken nokta, nesne zincirini kazara koparmamaktır: çekirdek operasyonun askıya alınması ya da durdurulması gerekmiyorsa onun çağrılması unutulmamalıdır. Soyut Decorator sınıfının kodlanması bu konuda yardımcı olur, ama somut dekoratörlerde metot yeniden tanımlanırken çağrı yine de unutulabilir.

3. Decorator Uygulama Örneği: Satış Makbuzu

3.1 Problem

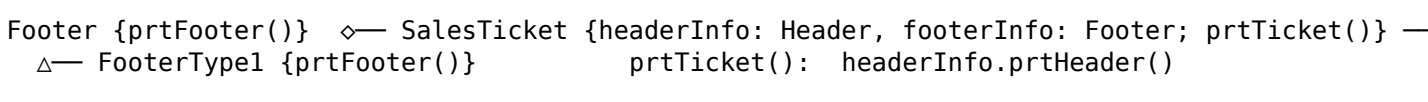
Bir e-ticaret programında `SalesOrder` sınıfı satış emirlerini yürütür; `process` metodu satış emrini işler. Satış emrinin makbuzu da basılmak isteniyor; makbuz ayrı bir `SalesTicket` sınıfıyla modellenir ve onun `prtTicket` metodu vardır. `SalesOrder`'ın makbuz basma metodu, ilişkili `SalesTicket` nesnesinin `prtTicket` metodunu çağırarak işi yürütür. Bu bir kullanma ilişkisi olabilir, `prtTicket` statik olabilir ya da farklı makbuz biçimleri varsa `SalesTicket`'ın alt sınıfları olabilir.

Sonra makbuzlara üstbilgi (header) ve altbilgi (footer) eklemek istendiği fark edilir.

3.2 İlk çözümler

Çözüm 1. Kullanma ilişkisine devam edilir: bir `Header` ve bir `Footer` sınıfı yazılır. `prtHeader` başlığı, `prtFooter` dipnotu yazar. `prtTicket` içinde önce header'ın `prtHeader` metodu çağrılır, sonra makbuzun basılması için gerekli işler yapılır, ardından footer'ın `prtFooter` metodu çağrılır. Fakat her makbuza header ya da footer basmak zorunda değilsek işin içine `if`'ler girer; örneğin `prtTicket`'a `hasHeader` ve `hasFooter` parametreleri eklenir ve bunlar doğruysa ilgili bilgi yazılır.

Çözüm 2 (ekrandan, 00:30:00). Farklı header ve farklı footer türleri varsa iki parametre yetmez. Farklı üstbilgi ve altbilgi türleri için ayrı kalıtım hiyerarşileri kullanılabilir:



```
  △── FooterType2 {prtFooter()}                          ...
                                                         footerInfo.prtFooter()
```

Slayta göre bu, henüz incelenmemiş Strategy kalıbının bir gerçeklemesidir. Hoca ayrıca farklı makbuz ailelerini düşünüp Abstract Factory, farklı header ve footer üreten bir Factory Method ya da karmaşık parçalardan oluşan bir varlığın oluşturulmasına yönelik Builder kalıbının da düşünülebileceğini söyledi; ama bunların genelde farklı ürün aileleri ve farklı seçenekler mantığında kullanıldığını belirtti. Bu örnekte Decorator mı Builder mı daha uygun olur, ya da ikisinin de avantajı var mı diye ayrıntılı bir inceleme yapılabileceğini, örneğin Decorator üzerinden yürütüleceğini ekledi.

3.3 Asıl gereksinim

Decorator'ı en uygun hâle getiren gereksinim şudur: aynı makbuzda birden fazla farklı üstbilgi ve/veya altbilgi basılması isteniyorsa. Slayttaki örnekler (ekrandan, 00:30:00):

```
HEADER1        HEADER1
HEADER2        Receipt
Receipt        FOOTER1
FOOTER2        FOOTER2
```

İlk makbuz iki header ve bir footer, ikincisi bir header ve iki footer içeriyor. "Yalnız header 1", "yalnız header 2" ve "header 1 ile header 2" için üç ayrı header türü kodlanabilir, ama bu durumda kopyala-yapıştır koddan kaçınılamaz; hoca bunun çok kötü bir yaklaşım olduğunu vurguladı. Slayta göre bu durumda gereksinimi ancak Decorator kalıbı karşılar: makbuz, istenen türde header ve footer'larla süslenince istenen sonuç elde edilir.

3.4 Önerilen çözüm

Önerilen çözümün sınıf şeması (ekrandan, 00:31:00):

```
Client ··> Component {+prtTicket()}  <──────── 1 ────────┐
             △── SalesTicket {+prtTicket()}               │
             △── TicketDecorator {-myComponent: Component; +prtTicket()} ◇ 0..1
                    △── Header1 {+prtTicket(), -prtHeader()}
                    △── Header2 {+prtTicket(), -prtHeader()}
                    △── Footer1 {+prtTicket(), -prtFooter()}
                    △── Footer2 {+prtTicket(), -prtFooter()}

TicketDecorator.prtTicket:  - check if myComponent is not null
                            - calls myComponent's prtTicket
Header2.prtTicket:  prtHeader();  TicketDecorator.prtTicket();
Footer1.prtTicket:  TicketDecorator.prtTicket();  prtFooter();
```

`SalesTicket` asıl işi (makbuzu basmayı) yapar. Soyut `Component` sınıfının soyut bir `prtTicket` metodu vardır; hoca `Component`'in arayüz de olabileceğini, ama `TicketDecorator`'ın arayüz olamayacağını yineledi. Farklı dekoratörler (`Header1`, `Header2`, `Footer1`, `Footer2`; gerekirse üçüncüleri de) `TicketDecorator`'ın alt sınıfları olarak kodlanır. Hepsinde `prtTicket` metodu vardır ve asıl yapılan, üst sınıfın `prtTicket`'ını çağırarak ilgili bileşenin `prtTicket` metodunun çalışmasını sağlamaktır; kendi işlerini de `prtHeader` ve `prtFooter` metotlarıyla yaparlar. Dekoratörün ek işlevlerini yapan bu metotlar, rastgele çağrılmasınlar diye private yapılabilir; istenirse "ancak bir bileşene bağlıysa çağrılsın" gibi ek denetimler de eklenebilir.

Sıranın önemi: header'da önce header basılır, sonra ilişkili nesneler zinciri kapsamında çekirdek iş (makbuzun basılması) yapılır. Footer'da ise önce silsile yoluyla çekirdek işin yapılması sağlanır, sonra footer basılır. Yani `prtHeader` `prtTicket` çağrısından önce, `prtFooter` ise sonra kodlanmalıdır; hoca bunun unutulmaması gerektiğini vurguladı.

> **Ders sonrası not:** Slayttaki şemada `TicketDecorator`'ın bileşenle ilişkisinin çokluğu `0..1` yazılmıştır. Hoca, dekoratör kurucusunda bileşeni parametre olarak aldığı için bileşenle ilişkili olmayan bir dekoratör oluşmadığını, çokluğun `1` olması gerektiğini söyledi ve bunu kendi slaytında düzeltilmesi gereken bir nokta olarak not aldı. Hoca şemayı örneği aldığı kitaptan aldığını belirtti.

3.5 Kod

`TicketDecorator` sınıfı `Component`'in alt sınıfıdır, mutlaka bir `Component` ile ilişkilidir ve kurucusunda bu bileşen verilerek oluşturulur. Çekirdek işin yapılması için `callTrailer` metodu vardır: nesne bir bileşenle ilişkiliyse onun işini yaptırır. Kurucu bir nesne beklese de belki kurucuya `null` verilerek oluşturulmuştur; bu `if` denetimi buna karşı ek bir önlemdir.

`Header1` ve `Header2` kodu (ekrandan, 00:35:40):

```
package decorator.example;
public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.callTrailer();
    }
}
public class Header2 extends TicketDecorator {
    public Header2 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 2 code here
        System.out.println(getClass().getName());
        super.callTrailer();
    }
}
```

Alt sınıfta üst sınıfın kurucusunu çağırmak zorunludur; bunun dışında kurucuda yapılacak ek bir iş yoktur. Header'ın yazma kodu yorum satırının yerine gelir. Hoca, hangi sınıftan bir nesnenin denetim akışında denetimi ne zaman aldığını göstermek için her dekoratörün kendi sınıf adını yazmasını istedi. Java'da `Object` sınıfında tanımlı `getClass()` metodu `Class` türünden bir nesne döndürür; bu nesne üzerinden, nesnenin türetildiği sınıfla ilgili birçok özelliğe erişilir. `Class` sınıfının `getName()` metodu sınıfın tam nitelikli adını (fully qualified name) döndürür; sınıf `decorator.example` paketinde olduğu için ekrana `decorator.example.Header1` yazılır. Ardından çekirdek işin yapılması için üst sınıfın metodu çağrılır. Gerçek bir uygulamada her dekoratörün basma kodu birbirinden çok farklı olurdu. `Footer1` ve `Footer2` de benzer şekilde yazılmıştır, ama onların `prtTicket` metotlarında önce çekirdek işin yapılması sağlanır, sonra footer yazma kodu gelir.

3.6 Zinciri kim kurar?

Client sınıfının kodlamasında önemli bir ayrıntı vardır: hangi tür makbuzun basılacağı çalışma anında nasıl belirlenecek? Bunu aslında makbuzu basacak kullanıcı belirler. Bir çözüm, Factory Method kalıbıyla hep belli türden makbuz basan nesneler oluşturmaktır. Daha basit bir çözüm, gerçek bir uygulamada kullanıcının bir açılır listeden (drop-down) hangisinin basılacağını seçmesidir. Nesneye dayalı birçok arayüz kütüphanesinde bu listeye yalnız metin değil herhangi bir sınıftan herhangi bir nesne konabilir; dolayısıyla listedeki bir öğe belirli bir dekoratör zinciriyle ilişkilendirilebilir.

Örnekte zincir, Factory Method'a bir adım yaklaşan bir `Factory` sınıfının `getComponent` metodunda kurulur (ekrandan, 00:40:00):

```
package decorator.example;
public class Factory {
    public Component getComponent () {
        Component myComponent;
        myComponent= new SalesTicket();
        myComponent= new Footer1(myComponent);
        myComponent= new Header1(myComponent);
```

```
        return myComponent;
        //VEYA: Aşağıdaki kod aynı etkiyi yapar
        //return(new Header1(new Footer1(new SalesTicket())));
    }
}
```

Önce bileşen olarak satış makbuzu oluşturulur. Sonra bu bileşen bir `Footer1` dekoratörüne verilir; `Footer1` de bir bileşen olduğu için artık yeni bir bileşen elde edilmiş olur. Sonra bu `Footer1`, aynı işaretçi yeniden kullanılarak `Header1` ile ilişkilendirilir. Böylece `Header1` dekoratörü `Footer1` dekoratörüyle, `Footer1` de `SalesTicket` çekirdek nesnesiyle ilişkili olur ve zincirin başlangıcı döndürülür. Aynı iş tek satırda da yazılabilir; hoca nesneye dayalı dillerde string işlemlerinin de genelde bu tür zincirlemeyle kodlandığını hatırlattı. Çalışma sonucu:

```
HEADER1
Receipt
FOOTER1
```

Hoca kodu Eclipse'te çalıştırırken `Header1`'in `Footer1`'den önce mi sonra mı verildiğinin bu örnekte önemli olmadığını gösterdi: önce `Header1`, sonra `Footer1` ile ilişkilendirildiğinde de çıktı aynıdır, çünkü biri header, biri footer'dır.

4. Decorator'da Zincirleme Sırası

Aynı türden iki dekoratör varsa, örneğin önce `HEADER1` sonra `HEADER2` ya da tersi basılacaksa, hangi header'ın hangi sırayla ilişkilendirildiği önem kazanır. Hoca bileşeni önce `Header2` ile, sonra `Header1` ile, sonra `Footer1` ile ilişkilendirip çalıştırdı:

```
myComponent = new SalesTicket();
myComponent = new Header2(myComponent);
myComponent = new Header1(myComponent);
myComponent = new Footer1(myComponent);
```

Çıktıda önce `HEADER1`, sonra `HEADER2`, sonra makbuz ve `FOOTER1` basıldı. Çalışma sırasını hatırlamak gerekir: header önce işini yapar, sonra isteği ilişkili bileşene yönlendirir. Dolayısıyla en üstte `HEADER1` basılsın isteniyorsa bileşen önce `Header2` ile, sonra `Header1` ile ilişkilendirilmelidir. Sıra tersine çevrilirse önce `HEADER2`, sonra `HEADER1` basılır. `Footer1` ise header'lardan önce de sonra da verilse sonuç değişmez, çünkü header ile footer'ın çalışma mantıkları farklıdır. Bu sonuçlar python ile kurulan bir benzetimle de doğrulandı.

Aradan sonra hoca, aynı tür mantıktaki dekoratörlerin basım sırasını görselleştirmek için iç içe (zincirleme) tek satır yazımın daha yararlı olduğunu gösterdi. Örneğin `new Header1(new Header2(new SalesTicket()))` yazımında bileşene daha yakın olan `Header2`'dir ve daha altta basılır; çıktı üstte `HEADER1`, altta `HEADER2` olur. Bazılarına zincirleme yazım karmaşık gelebilir, ama sırayı görmeyi kolaylaştırır.

Soru: Zincirleme bu şekilde kurulunca, örneğin uygulamada bir bilette `Header1` yukarıda, diğer bir bilette `Header2` yukarıda olsun istersek, zinciri her seferinde kodda değiştirmek mi gerekecek?

Cevap: Bunun için bir açılır liste ya da tıklanabilir bir menü gibi bir seçim mekanizması olmalıdır. Listenin her öğesine uygun biçimde zincirlenmiş bileşenin en üst ucu verilirse sorun kolayca çözülür. Hoca, sorunun karmaşık `switch-case` yapıları gerekip gerekmeyeceği kuşkusundan doğduğunu ve yerinde olduğunu, ama liste yaklaşımıyla böyle bir şeye gerek kalmayacağını söyledi.

5. Facade Kalıbı

Hoca kalıbın adının Türkçede "faça" (cephe, yüz) diye de geçtiğini, sözlükteki sesli telaffuzun "fasaad" biçiminde olduğunu belirtti.

5.1 Amaç ve motivasyon

Facade kalıbının amacı sisteme tek bir erişim noktası vermektir. Motivasyon şudur: farklı alt sistemler ve çeşitli işlemlerin başlangıç noktalarını oluşturan farklı sınıflar vardır. Örneğin 40 işlem varsa, onu bir sınıfta, onu bir başkasında, onu bir diğerinde olabilir. Böyle karmaşık bir arayüzdeki farklı erişim noktalarını tek bir noktada toplamak için Facade kullanılır.

Bunu zorlamak istersek, bütün alt sistem sınıfları bir pakette toplanır, Facade sınıfı da o paketin üyesi yapılır ve diğer sınıfların görünürlük kuralları kısıtlanır; hoca bunun biraz özel bir çözüm olduğunu ekledi. Asıl amaç ise şudur: alt sistem sınıfları doğrudan da kullanılabilir, ama bir istekçi geldiğinde hangi işin hangi sınıfın hangi metodunda olduğuyla uğraşmasın. Çok sayıda farklı sorumluluk tek bir Facade sınıfında toplanır; böylece istemciler alt sistemlerin alt düzey ayrıntılarından şeffaf biçimde soyutlanır.

5.2 Ne zaman kullanılır?

- Karmaşık bir alt sisteme basit bir arayüz sarmak istendiğinde.
- Alt sistemleri çeşitli katmanlara ayırmak istendiğinde.
- Bazı istekçilerin sistemin yalnız belli bir kısmıyla konuşmasını sağlayıp bağlaşımı azaltmak istendiğinde.

Farklı türde istekçilerin sistemin farklı alt sistemleriyle konuşması gerekiyorsa tek bir Facade sınıfı değil, birkaç Facade olur ve bir istemciye hangi Facade verilmişse ilgili alt sistem bileşenlerini onun üzerinden kullanır. Hoca bunun basit bir kalıp olduğunu söyleyerek örnek yapmadı.

6. Flyweight Kalıbı: Amaç ve Metin Editörü Örneği

Hoca “flyweight” sözcüğünün spor müsabakalarında rakiplerin ağırlık sıklet adlarından biri olduğunu (hafif sıklet, sinek sıklet gibi), kalıptaki anlamın biraz farklı olsa da bununla benzeşen bir yanı bulunduğunu belirtti.

6.1 Amaç

Kalıbın amacı, paylaşım yoluyla çok sayıda basit nesnenin kullanımının desteklenmesidir. Bu basit nesnelerin her birine flyweight denir.

6.2 Örnek ve sorun (ekrandan, 01:10:00)

Bir metin düzenleme programı yazılıyor. Programın font ve diğer biçimlendirme bilgilerine göre metni çok net göstermesi (excellent rendering) isteniyor; hangi font seçildiyse, italik mi değil mi, bu seçeneklere göre yazı ekranda çok güzel görünmelidir. Harfler grafik nesneleri olarak gösterilip en iyi sonuç alınmak isteniyor. Anlatımı kolaylaştırmak için hoca hep sabit genişlikli bir font (daktilo yazısı gibi) kullanıldığını varsaydı. Bu durumda bir sayfadaki metin, satırlar ve sütunlardan oluşan iki boyutlu bir matris gibi düşünülebilir; her gözde bir harf vardır.

Sorun şudur: her harf ayrı bir nesne örneği olarak ele alınırsa küçük bir belgede bile binlerce nesne olacaktır. Örneğin bir sayfada 50 tane `A` harfi varsa ekrana 50 ayrı `A` nesnesi mi basılacak? Bu 50 nesnenin tek farkı bulundukları satır ve sütun numarasıdır; hoca sayfadaki iki `A` harfini (altıncı satırın altıncı sütunu ile bir başka satırın dokuzuncu sütunu) göstererek bunların konum bilgileri dışında birbirinin aynı olduğunu anlattı. Her harfin ekrana çıkacak resmi saklanmak istendiğinden ve resim kendi içinde piksel satır ve sütunlarından oluştuğundan, bu nesneler bellekte ve ikincil saklama ortamlarında çok fazla yer kaplar. Slayta göre bu dersin üç sayfalık öneri formunda bile boşluklarla birlikte 5000 kadar harf bulunmaktadır.

6.3 Çözüm fikri

“Neyin değiştiğini bul ve onu sarmala” ilkesine göre: iki `A` harfi aynı bitmap’i paylaşır, farklı olan yalnız koordinatlarıdır. O hâlde değişen ve değişmeyen ayrı modellenir. Her satır ve sütunda yeni bir harf nesnesi oluşturmak yerine, her harf için tek bir harf nesnesi oluşturulur.

Bu bizim havuzumuzdur: A'dan Z'ye bütün harflerin (büyük ve küçük harfler için ayrı ayrı) bitmap grafikleri için bir havuz. Sayfanın ilgili satır ve sütununda `A` harfi varsa, her `A` için bellekteki aynı `A` nesnesine yalnız bir işaretçi verilir. Sayfa ekrana basılacağı zaman "burada `A` nesnesi var, bellekte şurada; git o resmi ekrana bastır" denir; sıra öbür `A`'ya gelince aynı resim ilgili yere yeniden bastırılır. Böylece harf nesnelerinin ortak bir havuzdan paylaşımlı kullanımı sağlanır ve bu paylaşılan nesnelere flyweight denir; kalıp adını buradan alır.

Çözümde bir metin içeriği olur; bunun satırları, sütunları ve karakterleri vardır. Herhangi bir satır ve sütuna bir karakter, ya da komple bir satır çizilebilir; ekranda sayfanın yalnız belli bir kısmı görünüyorsa yalnız o satırlar çizilir. Hoca bu kısmın tamamen uygulama mantığıyla ilgili bir tasarım olduğunu söyledi. Belli bir noktaya çizim yapılacağı zaman ilgili karakterin `draw` metodu havuzdaki harfin bitmap'ini alır ve verilen noktaya çizer.

7. Flyweight'in Yapısı ve Kullanım Durumları

7.1 İçsel ve dışsal durum

Kalıbın en önemli noktası, durum bilgisinin iki kategoriye ayrılabilmesidir. Dörtlü Çete kitabında bunlara intrinsic ve extrinsic denir:

- **İçsel (intrinsic) durum:** Flyweight nesnesinde her zaman saklanan durum bilgisidir. Örnekte harfin render edilmiş bitmap grafiği; aynı türden harfler arasında değişmez.
- **Dışsal (extrinsic) durum:** Harici bir nesneden flyweight'e aktarılan durum bilgisidir. Örnekte "bu harfi şu satır, şu sütun koordinatına bas" denirken verilen satır ve sütun bilgisi.

Bu ikisi ayrılabildiği için kalıp kullanılabilir. Ayrım yapılamıyorsa Flyweight'in hiçbir anlamı yoktur; o zaman bilgiyi birçok kez tekrarlamak, yani akla ilk gelen uygun olmayan çözümle devam etmek zorunda kalınır.

7.2 Kalıp yapısı

Slayttaki kalıp yapısı (ekrandan, 01:20:00):

```
FlyweightFactory {GetFlyweight(key)}  ◇── flyweights ──>  Flyweight {Operation(extrinsicState)
                                                             △── ConcreteFlyweight {Operation(
                                                             △── UnsharedConcreteFlyweight {Op
```

```
GetFlyweight(key):
    if (flyweight[key] exists) {
        return existing flyweight;
    } else {
        create new flyweight;
        add it to pool of flyweights;
        return the new flyweight;
    }
Client ──> FlyweightFactory
Client ──> ConcreteFlyweight
Client ──> UnsharedConcreteFlyweight
```

- **Flyweight:** İçsel durum bilgisi farklı somut flyweight'lerde çok farklı olacaksa, tanımlanması somut flyweight'lere bırakılır. Çok değişmeyecekse Flyweight sınıfı saf soyut ya da arayüz yapılmak yerine bazı somut üye alanlar ve somut metotlar içerebilir; ama her hâlükârda soyut bir sınıf olmalıdır. Yapılacak operasyon iki tür bilgiye ihtiyaç duyar: biri dışarıdan verilecek bilgi (dışsal durum, metoda parametre olarak verilir), diğeri nesnenin kendi içsel bilgisi (nesnenin durum bilgisi olarak zaten kendisindedir). İkisi bir araya gelince operasyon yapılabilir.
- **FlyweightFactory:** Bir sürü flyweight vardır, ama ihtiyaç duyulana nasıl erişilecek? Her flyweight'in tekil bir tanımlayıcısı olur; bu tanımlayıcıyı verip ilgili flyweight nesnesini geri alabileceğimiz bir factory ya da registry sınıfı ve bu sınıftan bir nesne olmak zorundadır. Kritik metot `GetFlyweight`'tir: ilgili anahtara sahip flyweight varsa onu döndürür; yoksa, istenirse, yeni bir flyweight oluşturup havuza ekler. İsteğe bağlı olarak anahtar-değer biçiminde bir ekleme (`set`) metodu da konabilir; örneğin yeni bir harf eklenecekse

onun grafiğini de vermek gerekir.
- **Client:** İhtiyaç duyduğu flyweight'leri fabrikadan alır ve kendi vereceği dışsal durumla ilgili operasyonları yürütür; flyweight nesneleri bu dışsal bilgiyi kendi içsel bilgileriyle birleştirerek işi yapar.

Hoca flyweight fabrikasının Singleton kalıbıyla da ilişkisi olduğunu, kalıplar arasındaki bu ilişkilere daha sonra bakılabileceğini söyledi. Şemada nesne işaretçisi yerine arayüz işaretçisi de görülebilir; hocanın örneğinde öyle yapılmıştır. Bir varyasyon olarak, operasyon metotları bir arayüzde tutulup somut flyweight sınıfları fabrikanın private iç sınıfı yapılırsa, o sınıftan dışarıda nesne elde edilemez; nesneye yalnız gerçeklediği public arayüz üzerinden erişilebilir.

Her flyweight'in paylaşılması gerekmez. Metin editörü örneğinde mantıklı olmasa da, bazı flyweight'ler paylaşılıp bazıları paylaşılmayacaksa (`UnsharedConcreteFlyweight`), istemci fabrikayla iletişim kurmadan paylaşılmayan somut flyweight nesnelerinden birine erişebilir; bu erişim de istenirse fabrika üzerinden yapılabilir. Çalışma anında havuzda çeşitli somut flyweight'ler bulunur; istemci önce istediği türü anahtar vererek fabrikadan alır, sonra onunla çalışır. Bir flyweight'e birden fazla istemci erişebilir; bir flyweight'e tek bir istemcinin erişmesi gerekiyorsa bunu da fabrika denetleyebilir.

7.3 Ödünleşim

Durum bilgisi içsel ve dışsal olarak ikiye ayrılmalıdır ve tasarruf fazla olsun diye durum bilgisinin önemli bir bölümünün çalışma anında hesaplanabilmesi tercih edilir. Ama nihayetinde flyweight kullanmakla kopya nesneler kullanmak arasındaki fark, yani tasarruf, az olacaksa Flyweight kurmaya gerek yoktur; fark edecekse kurmakta yarar vardır.

Hoca buna bir zaman-bellek ödünleşimi (time-memory trade-off) olarak bakmak gerektiğini söyledi. Flyweight'lerde saklanacak durum bilgisi azsa çok bilgi çalışma anında hesaplanır ve bu sefer hesaplama süresi söz konusudur. İçsel bilgi daha çoksa bellek gereksinimi artar. Bellekten tasarruf etmek mi, işlemden tasarruf etmek mi istendiğine göre karar verilir. Son olarak içsel kısmı oluşturan üyeler `final` tanımlanabilir.

7.4 Ne zaman kullanılır?

- Uygulama çok fazla nesne kullanıyorsa ve nesnelerin çokluğu nedeniyle saklama masrafı yüksek olacaksa. Bunlar ilk akla gelen nedenlerdir, ama kalıbın işe yaraması için başka şartların da sağlanması gerekir:
- Nesnelerin durum bilgisinin çoğu, en azından bir kısmı, dışsallaştırılabilmelidir. Aksi hâlde zaman-bellek ödünleşimi kapsamında kazanç olmaz.
- Dışsal durum ayrıldıktan sonra, nesnelerin birçok grubu yerine az sayıda paylaşılabilir nesne yetmelidir.
- Uygulama nesne kimliğine (object identity) bağımlı olmamalıdır.

7.5 Nesne kimliği örneği

Hoca nesne kimliğini bir web sitesinin ziyaretçi sayacıyla açıkladı. Sitenin o tarihe kadarki toplam ziyaret sayısı ile tekil ziyaretçi sayısı ayrı bilgilerdir. Bir kişi aynı siteye on ayrı anda girmişse ziyaret sayısını on artırmıştır. Sayaç 1 milyar gösterirken aslında 100 milyon kullanıcı her biri on kez girmiş olabilir. Tekil ziyaretçileri saymak için her ziyaretçinin tekil tanımlayıcısını, yani nesne kimliğini bilmek gerekir. Amaç yalnız ziyaret sayısını göstermekse nesne kimliğine ihtiyaç yoktur.

Flyweight'te belirli bir nesne kimliği yoktur, yalnız genel bir anahtar vardır; `A` harfinin anahtarı `A` karakteridir. Sayfaların birçok yerine basılan hep aynı `A` nesnesidir. Metin editöründe nesne kimliğine ihtiyaç olmadığı için kalıp kullanılabildi; ihtiyaç olsaydı Flyweight kullanılamazdı.

8. Flyweight Uygulama Örneği: Kimya Fabrikası

8.1 Problem ve ilk çözüm

Örnek Design Patterns Java Workbook kitabından alınmıştır. Bir kimya fabrikasının üretim bilgisayar sistemi oluşturuluyor. Fabrikada çok sayıda kimyasal madde üretilir; farklı kimyasal

maddeler bir araya getirilerek karışımlar da satılır. Aynı madde birden fazla karışımda yer alabilir.

İlk çözüm (ekrandan, 01:30:00):

```
KimyasalMadde                                          Karışım
  - gram : double                                        - isim : String
  - isim : String                                        - maddeler : LinkedList<KimyasalMade>
  - simge : String                                       + setIsim( String )
  - atomAgirligi : double                                + getIsim( ) : String
  + Kimyasal( isim : String, simge : String,             + maddeEkle( KimyasalMadde )
              atomAg : double )                          + tarifEt( ) : String
  + setGram( double )
  + getGram( ) : double                                Karışım ◇──> KimyasalMadde
  + getIsim( ) : double
  + getAtomAgirligi( ) : double
  + getMol( ) : double        // return gram/atomAgirligi;
```

Kimyasal maddenin durum bilgisi olarak atom ağırlığı, simgesi, ismi ve gramı tutulur; gram vardır çünkü aynı madde farklı paketlerde (10 gramlık, 20 gramlık kutu gibi) satılabilir. Kurucu gram dışındaki bütün alanları alır. Gram için setter ve getter, diğer alanlar için yalnız getter vardır, çünkü bir maddenin adı ve atom ağırlığı değişmez: kükürt kükürttür, altın altındır. Mol bilgisi gram bölü atom ağırlığıdır; hoca bunu kimyacılar için gerekli bir bilgi olarak koyduğunu, örnekte kullanılmadığını söyledi. Karışımın ismi ve içerdiği maddeler vardır; isim için setter ve getter, madde ekleme metodu ve karışımın tarifini (bir gram karışımda hangi maddeden kaç gram olduğunu) bir string olarak döndüren `tarifEt` metodu vardır.

> **Ders sonrası not:** Slayttaki şemada `getIsim( ) : double` yazılmıştır (ekrandan, 01:30:00); isim bir `String` olduğu için dönüş türü `String` olmalıdır. Kurucunun adı da `Kimyasal(...)` yazılmıştır; Java'da kurucu sınıfla aynı adı (`KimyasalMadde`) taşımalıdır. `LinkedList<KimyasalMade>` ifadesindeki `Made` de yazım hatasıdır.

Slayttaki örnek: 100 gr barutta 75 gr potasyum nitrat, 15 gr karbon, 10 gr kükürt bulunur. Bu çözüm bu tür bileşikleri destekler: bir karışım nesnesi oluşturulur, ismi barut olur, maddeleri ve gramajları eklenir. Ama karbon başka karışımlarda da, başka gramajlarla yer alacaktır. Yalnız gram bilgisi farklı, kalan bütün durum bilgisi aynı olan çok sayıda karbon nesnesi mi oluşturulmalı? Fabrikada çok sayıda karışım üretileceği için potasyum, kükürt ve diğer maddelerin de aynı biçimde çok sayıda kopyası olur; aralarındaki tek fark gramdır. Bu, belleğin verimli kullanılması değildir. Temel ilke yine "neyin değiştiğini bul ve onu sarmala"dır: değişen gram bilgisi madde sınıfından ayrılmalı, ayrı bir sınıfta yer almalıdır.

8.2 Ara tasarım: Bileşen sınıfı

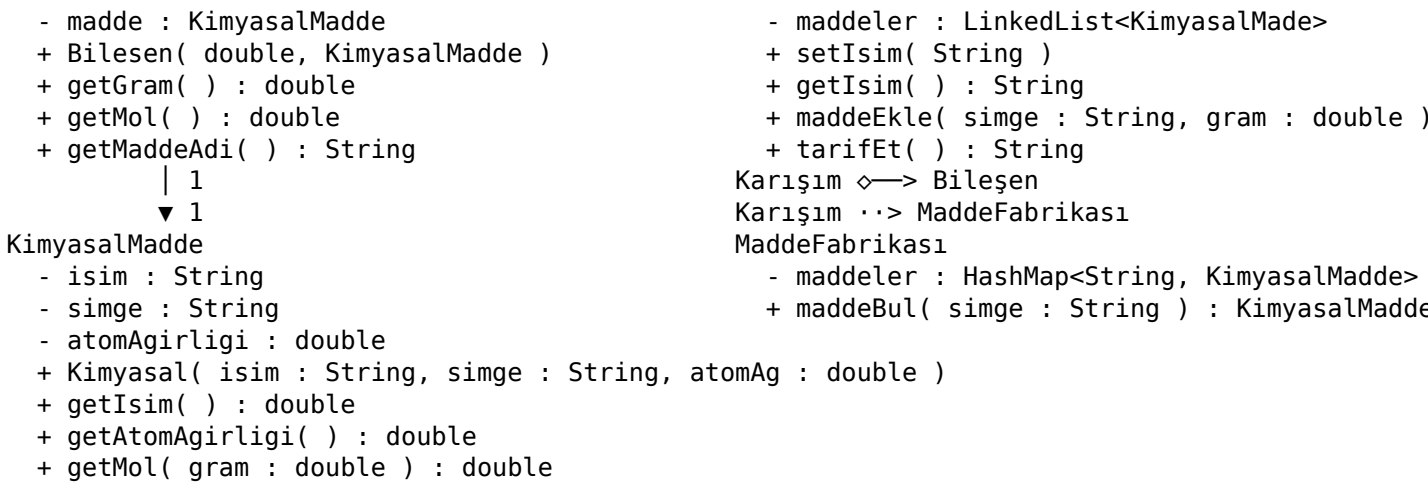
Yeni tasarımda karışım, bileşen ve kimyasal madde sınıfları vardır. `Bilesen` sınıfı çıkarıldı ve gram üye alanı kimyasal madde sınıfından bu yeni sınıfa taşındı. Hoca kod kusurları konusunda bu işlemin Extract Class (sınıf çıkarma) yeniden düzenlemesi (refactoring) olarak görüleceğini söyledi. Bileşen, gram açıklamasının hangi kimyasal maddeye ait olduğunu bilmelidir; bu yüzden bir `KimyasalMadde`'ye birebir sahiptir ve ilgili madde ile kaç gram yer aldığı kurucusunda verilir. Gram alanı ve gramla ilgili metotlar artık kimyasal madde sınıfında yoktur; kodun geri kalanı değişmez.

Hoca, tek bir varlığı esneklik için iki parçaya ayırınca ayrılan parçanın (bileşenin gramı) dışarıdan verilen, dışsal bir veri olduğunun dikkat çektiğini söyledi; diğer bilgiler hiç değişmiyor. Bir karışımda binlerce bileşen olacağı düşünülürse Flyweight kullanmaya değer. Bu ara tasarımın da fena olmadığını, ama Flyweight ile arasındaki farkı örnek kapsamında görmek istediklerini belirtti.

8.3 Flyweight ile çözüm

Flyweight ile çözümün sınıf şeması (ekrandan, 01:36:30):

```
Bileşen                                                Karışım
  - gram : double                                        - isim : String
```



Değişen kısım slaytta kırmızıyla işaretlenmiştir: karışımın madde ekleme metodu eskiden belli bir kimyasal madde nesnesi alıyordu; şimdi nesnenin kendisi yerine kimyasal maddenin tekil tanımlayıcısı olan simgeyi alıyor. Flyweight kalıbında ortak havuzdaki nesnelere tekil tanımlayıcıları üzerinden erişildiği için simgeye geçildi.

`MaddeFabrikasi` registry rolündedir. İlgili maddeler bir tablo yapısında, generic veri yapılarından `HashMap`'in anahtar (key) ve değer (value) bileşenlerinde saklanır. İlgili simgeye sahip maddeyi döndürmek için `maddeBul` metodu vardır; anahtarı alır ve veri yapısının "bu anahtara sahip nesneyi döndür" komutunu tek satırda çalıştırır, dönen nesneyi de iletir. Metot statik yapılabilir; böylece farklı yerlerden ortak kullanılabilir. Tasarımın geri kalanında bir değişiklik yoktur.

8.4 Kod

Kimyasal madde sınıfı gram dışındaki bilgileri tutan düz bir koddur. Madde fabrikası `HashMap`'i birkaç maddeyle doldurur: karbon (`C`), kükürt (`S`), potasyum nitrat (`KNO3`) ve sodyum nitrat (`NaNO3`). Hoca, fabrikanın potasyumu ve azotu ayrı ayrı değil, potasyum nitratı bileşik olarak ürettiğini varsaydıklarını söyledi. Maddeyi bulmak için veri yapısının `get` metodu kullanılır. Bileşen gram ve bir kimyasal madde içerir.

`Karisim` sınıfı (ekrandan, 01:39:00):

```
package dp.flyweight.solution3;
import java.util.*;
public class Karisim {
   private String isim;
   private LinkedList<Bilesen> maddeler = new LinkedList<Bilesen>();
   public String getIsim() { return isim; }
   public void setIsim(String isim) { this.isim = isim; }
   public void maddeEkle( String simge, double gram ) {
      maddeler.add( new Bilesen(gram, MaddeFabrikasi.maddeBul(simge)) );
   }
   public String tarifEt( ) {
      String tarif = isim;
      for( Bilesen madde : maddeler )
         tarif += "\n" + madde.getGram() + "gr " + madde.getMaddeAdi();
      return tarif;
   }
}
```

Madde eklemek için belli bir simgeli madde belli bir gramda eklenir: simge madde fabrikasının `maddeBul` metoduna verilir, dönen madde nesnesi gram parametresiyle birlikte `Bilesen` kurucusuna verilir, ortaya çıkan bileşen maddeler listesine eklenir. `tarifEt` metodu listedeki her bileşen için yeni bir satırda gramajı ve adı yazarak bir string oluşturur ve döndürür.

`MainApp` (ekrandan, 01:40:00 ve 01:41:15):

```java
package dp.flyweight.solution3;
public class MainApp {
    public static void main(String[] args) {
        Karisim karaBarut = new Karisim();
        karaBarut.setIsim("Kara barut");
        karaBarut.maddeEkle("NaNO3", 75);
        karaBarut.maddeEkle("C", 15);
        karaBarut.maddeEkle("S", 10);
        System.out.println( karaBarut.tarifEt() );
        Karisim temizBarut = new Karisim();
        temizBarut.setIsim("Temiz barut");
        temizBarut.maddeEkle("KNO3", 75);
        temizBarut.maddeEkle("C", 20);
        temizBarut.maddeEkle("S", 25);
        System.out.println( temizBarut.tarifEt() );
    }
}
```

Derste Eclipse'te çalıştırılan sürümün çıktısı (ekrandan, 01:41:15):

```
Kara barut
75.0gr Sodyum Nitrat
15.0gr Karbon
10.0gr Kükürt
Temiz barut
75.0gr Potasyum Nitrat
20.0gr Karbon
25.0gr Kükürt
```

> **Ders sonrası not:** Slayttaki `MainApp` kodunda temiz barut için `C` 15 ve `S` 10 yazılıdır (ekrandan, 01:40:00); derste çalıştırılan Eclipse sürümünde ise `C` 20 ve `S` 25 kullanılmıştır (ekrandan, 01:41:15). Çıktı ikinci sürüme aittir. Gramajlar örnek amaçlıdır; slayttaki "100 gr barutta 75 gr potasyum nitrat" ifadesine karşın koddaki "Kara barut" sodyum nitratla, "Temiz barut" potasyum nitratla kurulmuştur.

8.5 Kazanç

Hoca tasarımı yeniden göstererek şunu söyledi: bu yolla elde edilen tasarruf, bir kimyasal madde ne kadar çok farklı bileşende yer alıyorsa o kadar büyüktür, çünkü maddenin her bileşen için ayrı kopyası oluşturulmaz. Öte yandan `Bilesen`'li ara tasarım ile Flyweight'li tasarım arasında çok da büyük bir fark yoktur. Flyweight uygulanırken oluşturulan madde fabrikası, farklı maddelere ulaşmak için merkezî ve tekil bir erişim noktası sunar; bu da bir adım daha tasarruf sağlar. Karışımlarda maddeler iki tasarımda da bir `LinkedList` içinde tutulur, ama Flyweight'li tasarımda madde nesnelerinin kopyası tutulmaz. Hoca Flyweight'in, akla gelen düzgün tasarıma göre biraz daha fazla tasarruf sağladığını ve tercih edilebileceğini ya da edilmeyebileceğini, ama "neyin değiştiğini bul ve onu sarmala" ilkesine göre bölümleme yapılabildiği için önemli bir tasarruf elde edildiğini söyledi.

9. Proxy Kalıbı

9.1 Amaç ve örnek

Proxy, vekil anlamına gelir. Kalıbın amacı, bir nesneye erişimi denetlemek için o nesnenin yerine geçecek başka bir nesneyi araya koymaktır.

Örnek: ekrana bir resim çizmek zaman alıcı bir işlem olabilir. Hoca, eskiden uzun sayfalı bir metin dosyasında dolaşırken resimlerin ekrana gelmesinin bir süre aldığını, ilk başta resmin yerinde kapsayıcı bir dikdörtgen ve sol üstte küçük bir simge göründüğünü, yarım saniye sonra resmin çıktığını hatırlattı. İnternet yavaşken resim ağırlıklı bir web sitesinde de benzer durum yaşanabilir. Bu yüzden bir kelime işlem programı büyük resimleri hemen belleğe yüklemeyebilir. Örneğin çok sayfalı, içinde şekiller ve denklemler bulunan bir belge açılırken sistem, yanıt süresini (response time) düşük tutmak, belki de bellek tasarrufu yapmak için belgedeki bütün

grafikleri hemen belleğe yüklemez; ilgili sayfa ekranda gösterilmek istendiğinde yükler ve çizer. Akıllı bir ön tamponlama özelliği varsa, kullanıcı o sayfaya birkaç sayfa kala yüklemeye başlamaya da karar verebilir.

Bunun mantığı şudur: metin belgesi resimleri doğrudan tutmak yerine bir `ImageProxy` nesnesine işaretçi tutar. Sayfanın ekranda çizilmesi gerektiğinde proxy, dosya yolunu bildiği resmi alır ve ekrana çizer.

9.2 Örnek çözümün sınıf şeması

Slayttaki örnek çözüm (ekrandan, 02:00:00):

```
DocumentEditor ──> Graphic {Draw(), GetExtent(), Store(), Load()}
                     △── Image {Draw(), GetExtent(), Store(), Load(); imageImp, extent}
                     △── ImageProxy {Draw(), GetExtent(), Store(), Load(); fileName, extent}
                            ── image ──> Image
```

```
ImageProxy.Draw():
    if (image == 0) {
        image = LoadImage(fileName);
    }
    image->Draw()

ImageProxy.GetExtent():
    if (image == 0) {
        return extent;
    } else {
        return image->GetExtent();
    }
```

Belge editörü çeşitli grafikler içeren bir metinle çalışır. Soyut `Graphic` sınıfının çizilmesi, boyutunun bilinmesi, saklanması, yüklenmesi gibi işlemleri vardır. `Image` bunun somut alt sınıfıdır; ilgili metotlar orada somut olarak yer alır, resim verisi ve konum bilgileri de orada tutulur. `ImageProxy` de `Graphic`'in somut bir alt sınıfıdır ve bütün metotlara sahiptir; ama yaptığı iş aslında metot çağrısını ilgili nesneye iletmektir. `Draw`'da resim (`image`) `null` ise, yani henüz yüklenmemişse, resim ilgili yerden belleğe yüklenir, sonra çizilir; resim zaten yüklenmişse doğrudan ona çizdirilir. `GetExtent`'te resim yüklenmemişse proxy yüklemeyi de isteyebilir ya da bir iki piksel gibi varsayılan bir değer döndürebilir; bu, tasarımcıya kalmıştır. Yüklenmişse asıl nesnenin boyut bilgisini döndürür. Hoca `extent`'in en-boy bilgisi ya da grafiğin sayfada hangi koordinattan başladığı olabileceğini, hangisinin kastedildiğini hatırlayamadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Slayttaki `image == 0` ve `image->Draw()` gösterimi büyük olasılıkla GoF kitabından alınmış C++ sözdizimidir (GoF kitabındaki örnekler C++ ve Smalltalk ile yazılmıştır); `0` burada boş işaretçiyi, Java'daki `null`'u temsil eder.

9.3 Kalıp yapısı

Gerçek bir varlık (RealSubject) vardır. Proxy'nin bu nesnenin yerine geçebilmesi için gerçek varlığa bir arayüz tanımlamak şarttır; varlığın yetenekleri bu `Subject` arayüzünde listelenir. Proxy de bütün bu metotları gerçekler, ama birebir kopyalamaz: yukarıdaki mantıkta olduğu gibi nesne yüklenmemişse yükle gibi ek kodlar ekleyip sonra işi asıl nesneye yaptırabilir; öncesine ve/veya sonrasına çeşitli işler (örneğin günlük tutma) eklenebilir. Bellekte gerçek varlık bir vekil nesnesiyle ilişkilidir, istemci de vekille ilişkilidir; işler vekil üzerinden yürür.

9.4 Diğer kullanım durumları

- Nesneye basit bir işaretçiden daha fazlası gerekiyorsa; farklı işlemler proxy'ye kodlanabilir.
- İlgili varlık başka bir adres uzayında yer alabilir. Örneğin 50 bilgisayardan oluşan dağıtılmış bir sistemde erişilmek istenen varlık bu bilgisayarlardan herhangi birinde depolanıyor olabilir, ama sistemdeki her bilgisayarda o varlık için bir vekil bulunabilir. Vekil

sayesinde nesne, sanki kendi bilgisayarımızda ve kendi adres uzayımızdaymış gibi, ağ üzerinde başka bir bilgisayarda olmasının karmaşıklığından bağımsız kullanılır. Varlığın durum bilgisi değişecekse ve ağdaki herhangi bir bilgisayardan eşzamanlı değişiklik talepleri gelebiliyorsa, bütün senkronizasyon sorunlarını arka tarafta proxy başka mekanizmalarla konuşarak istemciden soyutlar.

Ek bilgi (derste anlatılmadı): GoF kitabında proxy'ler büyük olasılıkla kullanım amacına göre şöyle adlandırılır: başka bir adres uzayındaki nesneyi temsil eden remote proxy, pahalı nesneyi gerektiğinde oluşturan virtual proxy (yukarıdaki `ImageProxy`), erişim haklarını denetleyen protection proxy ve nesneye erişirken ek işler yapan smart reference. Derste dağıtılmış sistem örneği remote proxy'ye, resim örneği virtual proxy'ye karşılık gelir.

10. Proxy Uygulama Örneği: Pay-per-view Yayın Sunucusu

10.1 Problem

İzlediğin kadar öde (pay-per-view) sunucusuna gelen taleplerin güvence altına alınması istenir. Bir yayın sunucusu, kendisine verilen adrese (istekçi televizyon ekranına) istenen programı gönderir, belki akış (stream) olarak. Göndermeden önce videoyu talep eden istekçinin abonelik ücretini ödeyip ödemediği, pay-per-view ise hesabına önceden kredi yükleyip yüklemediği denetlenmelidir. Örneğin bir otelde ek hizmet olarak otelin video sisteminden bir video seçip izlemek ücrete tabi olabilir.

Elimizde temel işleve sahip bir sunucu varsa ve bu istekleri ödeme denetimi, hatta ödeme yapılmamışsa ödemeyi alma gibi ek işlevlerle zenginleştirmek istiyorsak Proxy uygun bir kalıptır. İstekçilerin set üstü kutularında ya da akıllı televizyonlarında çalışan kodun kaynağına erişim yetkimiz olmayabilir. Bu kodun tek yaptığı belli bir sunucunun metotlarını çağırmaksa, gerçek varlık olan yayın sunucusunun yerine proxy konduğunda istekçi farkında bile olmadan çalışmaya devam eder. Hoca bu tür proxy'lere literatürde protection proxy dendiğini belirtti.

Senaryoda yayın sunucusunun zaten bir arayüzü olduğu varsayılır. Kaynak kodu değiştirilemiyorsa ve böyle bir arayüz yoksa senaryo mantıklı olmaz, çünkü yeni bir `StreamSource` arayüzü ekleyip sunucunun onu gerçeklemesi için sunucunun kodunu değiştirmek gerekir. Ya da kod değiştirilebiliyordur ama sunucu kodunu bu ek yeteneklerle değiştirmek istenmiyordur; örneğin bazı müşterilere yalnız çekirdek ürün olarak yayın sunucusu, bazılarına koruma, pay-per-view gibi yetenekleri olan üst paket satılıyordur.

10.2 Tasarım

Slayttaki çözüm (ekrandan, 02:10:00):

```
«interface» StreamSource
  + stream(programID, destination : String) : boolean

StreamServer  ┄┄▷ StreamSource
  + stream(programID, destination : String) : boolean

StreamProxy  ┄┄▷ StreamSource,   StreamProxy ──1──> StreamSource
  - original : StreamSource
  + StreamProxy(original : StreamSource)
  + stream(programID, destination : String) : boolean
  + checkPayment(programID, destination : String) : boolean

StreamDestination  ──1──> StreamSource
  - server : StreamSource
  - address : String
  + StreamDestination( address : String )
  + getAddress( ) : String
  + setServer( StreamSource )
  + makePayment( double ) : boolean
  + requestProgram( programID : String ) : boolean
```

`MainApp ──> StreamDestination,  MainApp ──> StreamServer,  MainApp ──> StreamProxy`

- `StreamProxy` çözüm kapsamında eklenir. Orijinal sunucuyu somut sınıfı türünden değil, arayüzü türünden tutar; hoca somut sınıfa birebir sahiplik ilişkisi kurup bağlaşımı artırmaya gerek olmadığını, proxy'nin sunucunun diğer özelliklerini bilmesine gerek olmadığını söyledi. Orijinal kaynak kurucuda verilir. `stream` belirli bir programı belirli bir istekçiye gönderir ve başarılıysa `true` döner. `checkPayment` belirli bir hedefin, belirli bir programın izlenebilmesi için yeterli krediyi yükleyip yüklemediğini denetler; yeterli bakiye varsa `true`, yoksa `false` döndürür.
- `StreamDestination`, hizmetin abonelerini ya da otel odalarındaki akıllı televizyonları ve set üstü kutuları modeller. Bir `StreamSource` ile ilişkilidir; bu ilişki arayüz üzerinden kurulur, çünkü ona sunucunun kendisi değil proxy verilecektir. Adres getter'ı, sunucuyu değiştirmek için setter, ödeme yapma metodu (bir bakiye alanı da olabilirdi) ve belirli bir programı sunucudan isteyen, istek olumlu karşılanırsa `true` dönen `requestProgram` metodu vardır.

10.3 Kod

Kodlar iskelet olarak yazıldı; hoca video akışının nasıl yapılacağını bilen birinin bunları doldurabileceğini, örneğin bir tasarım örneği olarak verildiğini söyledi. Arayüzde yalnız işle ilgili metot tanımlıdır. `StreamServer.stream` programı hazırlayıp göndermek için gerekli işleri yapıyormuş gibi yazılmıştır. `StreamDestination`'da ödeme metodu yalnız "ödeme cihazıyla entegrasyon kurulacak" anlamında bir yorum içerir. `MainApp`'te bir sunucu, bir proxy ve bir istemci oluşturulur ve istemciye sunucu olarak proxy verilir. Çıktısı fark yaratacak bir şey göstermediği için çalıştırılmadı.

11. Davranışsal Kalıplara Giriş

Proxy ile yapısal kalıplar bitti ve son kalıp türü olan davranışsal (behavioral) kalıplara geçildi. Davranışsal kalıpların ortak noktası, nesnelerin davranışlarını çalışma anında değiştirebilmelerini sağlamaktır.

Hoca yalnız bu tanıma bakılırsa Decorator'ın da davranışsal sayılabileceğini belirtti: Decorator bir nesneye çalışma anında sorumluluklar ekler, hatta eklenen sorumluluk özgün sorumluluğu tamamen iptal bile edebilir. Yani yapısal kalıplar da bir nesneye esneklik sağlayıp onun çalışma biçimini çalışma anında değiştirebilir. Ama ana katkısı bakımından Decorator yapısal bir kalıptır; birazdan görülecek Chain of Responsibility ise davranışsal bir kalıptır. Hoca kalıpları bu üç türden birine kesin biçimde yerleştirmeyi sevmediğini, ama konuyu bu şekilde ilerletenlerin çok olduğunu söyledi. Bazı sınavlarda "şu kalıp hangi gruba girer" türünden sorular olur; hoca böyle sorular sormadığını, kalıpların hangi gruba girdiğinin ezberlenmesini kesinlikle istemediğini belirtti.

12. Chain of Responsibility Kalıbı

12.1 Amaç

Birbiriyle ilişkili bir nesneler zinciri ya da nesne hiyerarşisi vardır; ya hâlihazırda vardır ya da iş mantığı kapsamında kurulması gerekir. Zincirdeki bir nesne bir işi gereğince yapamayabilir: hiç yapamayabilir ya da kısmen yapabilir. Bu durumda çağrının, işi tam yapacak nesneye yöneltilmesi en doğrusudur. Chain of Responsibility bu biçimde çalışan bir kalıptır.

Hoca bir ayrımı vurguladı: nesnenin mesajın gereğini yerine getirememesi, o metodun onda tanımlı olmamasından kaynaklanmaz. Aynı metot farklı nesnelerin hepsinde vardır. Ama nesne gerekli yetkiye ya da bilgiye tam olarak sahip değildir ve aynı metot çağrısını yetkiye ya da bilgiye sahip olan nesneye iletir. Tanımlı olmayan bir metodu çağırınca bir şeylerin olduğu bir çalışma biçimi söz konusu değildir; bu çoğu dilde yoktur ama ilginç biçimde Smalltalk'ta vardır.

> **Ek bilgi (derste anlatılmadı):** Hocanın Smalltalk için kastettiği büyük olasılıkla şu mekanizmadır: bir nesne anlamadığı bir mesaj aldığında `doesNotUnderstand:` mesajı tetiklenir ve bu metot yeniden tanımlanarak mesaj başka bir nesneye iletilebilir.

Chain of Responsibility ise bu mekanizmaya dayanmaz; zincirdeki her nesnede aynı metot tanımlıdır.

12.2 Örnek: bağlama duyarlı yardım

Hoca örneği eski uygulamalardaki diyalog kutularıyla anlattı. Bir diyalog kutusunda birçok bileşen vardır. Diyalog kutusunun genel yardım dosyasında merak edilen parçanın ayrıntısını aramak yerine, doğrudan o parça hakkında yardım istenebilir: önce soru işaretine tıklanır, fare imleci değişik bir şekil alır, sonra örneğin yazı tipi alanına tıklanınca yalnız yazı tipi değiştirmeyle ilgili yardım gösterilir. Hoca bunu PowerPoint'te (kılavuz ayarları diyaloğunda) denedi, ama soru işaretinin doğrudan genel yardım dosyasını açtığını gördü; eskiden bütün uygulamaların bu tür soru işaretleri koyduğunu, şimdilerde böyle bir uygulamaya rastlamadığını söyledi.

Örnek, zaten birbiriyle ilişki içinde olan varlıklardan oluşur: ana uygulamanın bir şeridi (ribbon), şeridin içinde çeşitli alt birimler vardır; birine tıklanınca bir diyalog kutusu açılır; diyalog kutusunda bölümler ve düğmeler, bölümlerin içinde de açılır liste, onay kutusu gibi denetimler vardır. Bir hiyerarşi, bir birliktelik zaten mevcuttur. Soru işaretine tıklanıp bir denetime tıklanınca oranın yardımı gelmelidir. Program ekibi o denetime özel yardım hazırlamamış, ama diyalog kutusuna yönelik genel bir yardım hazırlamışsa diyalog kutusunun yardım metni; diyalog kutusuna özel bir yardım da yoksa programın genel ana yardım sayfası açılmalıdır.

Yardım çağrısı yapılan bileşen kendine özel bir yardıma sahipse onu gösterir; aksi hâlde istek bir üst bileşene iletilir. Bu iletim, istek bir nesne tarafından yanıtlanana ya da zincirin ucuna gelinene kadar sürer. Her bileşen hangi üst bileşenin içinde yer aldığını uygulama mantığı gereği zaten bilir. Yani daha özelden daha genele bir yardım zinciri vardır.

Mesajın iletilebilmesi için bileşenlerin ortak bir arayüze sahip olması gerekir; büyük olasılıkla zaten vardır, çünkü bu bileşenler bir Composite kalıbı gerçeklemesinin parçalarıdır. Ortak bir arayüz ya da ortak bir üst sınıf yoksa `if`, `else`, `switch`, `case`, tür denetimi ve tür dönüşümü gibi pek güzel olmayan kodlar yazmak gerekir. Bu hâliyle bir düğmenin kendisine tanımlı bir yardım dosyası yoksa `HandleHelp` metodu içinde bulunduğu diyaloğun `HandleHelp` metodunu çağırır; o da özel içeriğe sahip değilse ana uygulamanın `HandleHelp` metodu çağrılır ve uygulamanın ana yardımı gösterilir.

Tasarımda `HelpHandler` soyut bir sınıf olur, çünkü nesne kendisi yanıt veremezse kime ileteceği bilgisi tutulmalıdır; bileşenler bir Composite ise bu bağlantı zaten vardır. Soyut bir widget sınıfı ve somut widget'lar bulunur; hepsi yardım edebilir, edemiyorsa ilişkili olduğu nesnenin yardımını çağırır.

12.3 Kalıp yapısı

Kalıp, daha sade bir sınıf tasarımıyla önerilir (ekrandan, 02:24:00):

```
Client ──> Handler {HandleRequest()} ── successor ──> Handler
              △── ConcreteHandler1 {HandleRequest()}
              △── ConcreteHandler2 {HandleRequest()}

Nesne düzeyinde:
aClient.aHandler ──> aConcreteHandler.successor ──> aConcreteHandler.successor
```

Kalıp bileşenleri (slayttan):

- **Client:** Yanıtlanacak mesajın kaynağı.
- **Handler:** Yanıtlanacak mesajı tanımlayan arayüz. Zincir bağlantısını (successor) da gerçekleyebilir (soyut sınıf olarak).
- **ConcreteHandlerX:** Mesajı yanıtlar veya bağlantılı olduğu bir başkasına iletir.

Hoca zinciri kuran `successor` üyesinin önemli olduğunu, onsuz kalıbın oturtulamayacağını; zaten varsa var olduğunu, yoksa eklenmesi gerektiği için şemada gösterildiğini söyledi. Çeşitli handler'lar birbiriyle ilişki içindedir; isteği en iyi yanıtlayabilecek olan yanıtı verir, veremeyen ilişkili olduğu bir üst makama iletir.

12.4 Ne zaman kullanılır, zayıf yönü

Bir mesajı yerine getirebilecek birden fazla nesne varsa ve en iyi yanıt verecek nesne, hele ki önceden ya da çalışma anında bilinemiyorsa bu kalıp kullanılır.

Kalıbın en zayıf yönü, zincirin doğru yapılandırılması gerekliliğidir. Aksi hâlde bir metot çağrısının yanıtlanacağının garantisi yoktur; doğru bir sonlandırma gerekir. Çağrı bir nesneye gelir, o yapamaz, ilişkili olduğuna iletir; o da bir sonrakine iletir ve zincirin sonuna gelinir, ama hâlâ düzgün yanıt verilmemiştir. Metodun bir iş yaptırması gerekir, böyle olmamalıdır.

Zincir hâlihazırda yoksa, örneğin bir Composite yapılmamışsa, zincir oluşturma metotları en uygun yer olan soyut ortak `Handler` üst sınıfına kodlanır ve kalıtımla alt sınıflara aktarılır. Yalnız yardım değil başka metot çağrılarının da aynı biçimde davranması isteniyorsa `HandleRequestX` gibi n adet farklı metot olabilir; hepsi aynı mantıkla çalışır: "ben yapabiliyor muyum, yeterince yerine getirebiliyor muyum; getiremiyorsam ilişkili olana ilet." İstenirse kalıtım yoluyla da kurulabilir, ama hoca bunu pek tercih etmediğini, mümkün olduğunca kalıtım yerine parça-bütün (sahiplik) ilişkilerini kullanmaya çalıştıklarını söyledi.

13. Chain of Responsibility Uygulama Örneği: Reklam Yönetim Yazılımı

13.1 Problem

Hoca bu örneği kendisinin uydurduğunu söyledi. Bir reklam yönetim yazılımı hazırlanıyor. Portföyde reklam gösterilebilecek çeşitli web sitesi alanları var; bu alanlarda çeşitli kampanya kuralları tanımlanıp onlara göre reklamlar gösterilmek isteniyor. Slayttaki problem (ekrandan, 02:30:00):

- **Kampanya 1:** Spor sitesinin 18-39 yaş arası erkek ziyaretçilerine traş bıçağı reklamı göster, 18-39 yaş arası kadın ziyaretçilerine fitness salonu reklamı göster, diğer ziyaretçilere spor mağazasındaki indirimin reklamını göster.
- **Kampanya 2:** Haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamını göster, aksi hâlde spor sitesinin kampanyasını aynen uygula.
- **Kampanya 3:** Ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından birini göster.
- …

Hoca "spor mağazasındaki indirim" ifadesinin kendi kampanya kuralı kavramından farklı bir şey olduğunu, bunun spor mağazasının bir indirim kampanyası olduğunu ve onun reklamının gösterileceğini açıkladı. Kampanya 2'de bir kampanyanın yeniden kullanımı söz konusudur; farklı bir şey yazılsaydı benzer biçimde ayrı bir kampanya tanımlanırdı.

Hoca bunun Decorator'a benzetilebileceğini, ama Decorator olmadığını vurguladı: Decorator'da zincirdeki her bileşen kendi işini yapar, birden fazla iş yapılır. Chain of Responsibility'de ise yalnız biri yanıt verir; istenen koşul sağlanmıyorsa bir sonrakine iletilir. Bu yüzden bu probleme Decorator kalıbı uymaz.

13.2 Tasarım

Hoca sınıf şemasının zamanında Eclipse'e kurduğu bir eklentiyle otomatik oluşturulduğunu, sadeleştirmek için epey uğraştığını söyledi. Ana uygulamada kampanyaları çalıştıran metotlar vardır. Belli bir kullanıcıya belli bir reklam gösterileceği için kullanıcı ve reklam sınıfları bulunur.

- **`User`:** Cinsiyet, gelir düzeyi ve yaş için üye alanları vardır. Kurucu hepsini atar, hepsi için getter vardır. `toString` metodu kullanıcı hakkında bilgi içeren bir string döndürür. Kampanyalarda "kullanıcı şu yaş ile şu yaş arasında mı" sorusu sorulduğu için kullanıcının yaşının belli sınırlar içinde olup olmadığını sınayan `yasDahilMi` metodu da eklenmiştir; hoca bunu kolaylık için koyduğunu, yoksa bu denetimin kampanyanın ilgili metodu içinde yazılması gerekeceğini söyledi. Kullanıcının adı tutulmaz.
- **`Cinsiyet`:** Kadın ya da erkek değerlerini alan bir enum.
- **`GelirDuzeyi`:** A, B, C değerlerini alan basit bir enum. Hoca hangi gelirin hangi gruba girdiğini ayrıca modellemek istemediğini, ekonomik koşulların sürekli değiştiğini belirtti.

- **Reklam:** Soyut sınıftır. Bir reklam bir `Banner` ile ilişkilidir (banner çizimi yapılmadı). Reklam kullanıcıya uygun değilse sonrakini göstermek, yani zinciri kurmak için `sonraki` alanı vardır; şemada `Reklam`'dan kendisine giden bir ok olmalıdır, şemada gösterilmemiştir. `reklamUygunMu` ve `reklamGoster` soyut bırakılmıştır, çünkü reklamın gösterilme biçimi ne reklamı olduğuna göre değişir: normal resim, flash animasyon, HTML5 animasyon, video vb.
- **Somut reklamlar:** `IndirimSporMagaza`, `TrasBicak`, `OtoDergi`, `FitnessSalonu`. Başka kampanyaların gerektirdiği reklamlar kardeş alt sınıflar olarak eklenebilir.

13.3 Reklam sınıfı ve zincir mekanizması

`Reklam` sınıfının kodu (ekrandan, 02:36:30):

```
public Reklam(Banner banner, Reklam sonraki) {
    this.banner = banner; this.sonraki = sonraki;
}
public abstract boolean reklamUygunMu ( User u );
public abstract void reklamGoster( User u );
public final void reklamKotar( User u ) {
    if( reklamUygunMu(u) ) {
        System.out.println( this.getClass().getName()
            + "->" + u );
        reklamGoster(u);
    }
    else if( sonraki != null ) {
        System.out.println( this.getClass().getName()
            + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
}
```

Kurucu banner'ı ve sonraki reklamı atar. Hoca daha önce söylediği ilkeyi burada uyguladığını belirtti: zincirin bileşenleri kendi aralarında hâlihazırda ilişkili değilse, ilişkinin kurulması ve yönetilmesi en uygun yer olan üst sınıfta yapılır. Alt sınıflar bu mekanizmaya müdahale edemesin diye `reklamKotar` metodunun `final` tanımlanmasında büyük yarar vardır.

Çalışma biçimi şudur: reklam ilgili kullanıcıya uygunsa reklam gösterilir; gösterim ilgili alt sınıf sayesinde gerçekleşir. Uygun değilse ve sonraki bir bileşen varsa istek ona iletilir. Hiçbiri yoksa yapılan bir şey yoktur. Varsayılan bir reklam gösterilecekse bunu buraya sabit kodlamak doğru değildir; bunun yerine somut reklam sınıflarının arasına varsayılan bir reklam sınıfı eklenebilir. Hoca, reklam ajansı olarak bu varsayılan reklamda kendi ajansının reklamını yapabileceğini, böylece iletim zincirinin en azından bir mesaj göstererek sonlanmasının garanti edileceğini söyledi. Öte yandan kampanya kuralları zaten "şu kritere uyuyorsa şunu göster, olmadıysa en son şunu göster" diye bittiği için bu zorunlu değildir, ama farklı olasılıkları düşünüp ona göre davranmakta yarar vardır.

Reklam isteklerinin yönlendirilme sırasının doğru kodlanıp kodlanmadığını görmek için yine Java'nın yansıma (reflection) yeteneklerinden yararlanılır. Reklam kullanıcıya uygunsa ve gösterilecekse reklam sınıfının adı, ardından `->` ve kullanıcının `toString` çıktısı (cinsiyet, gelir düzeyi, yaş) yazılır. Reklam gösterilmemişse önce reklamın kendi adı, sonra "bunu ilettim" anlamında `->` ve sonraki reklamın adı yazılır. Sınıf adları, Decorator örneğindeki gibi `getClass().getName()` ile tam nitelikli olarak alınır.

13.4 Somut reklamlar

`TrasBicak`, `Reklam`'ın alt sınıfıdır; sonraki reklam kurucudan iletilir. `reklamUygunMu` metoduna kullanıcı parametre olarak gelir ve kampanyadaki kural düz bir biçimde kodlanır: 18-39 yaş arasındaki erkek kullanıcılara uygundur. `FitnessSalonu` (ekrandan, 02:41:00):

```
package dp.chainOfResponsibility.example;
public class FitnessSalonu extends Reklam {
```

```
    public FitnessSalonu(Banner banner, Reklam sonraki) {
        super(banner, sonraki);
    }

    public void reklamGoster(User u) {
        //Gerekli komutlar
    }

    public boolean reklamUygunMu(User u) {
        if( u.getCinsiyet() == Cinsiyet.KADIN )
            if( u.yasDahilMi(18, 39))
                return true;
        return false;
    }
}
```

Kullanıcı uygunsa `reklamKotar` gösterme metodunu çağırır. Diğer ziyaretçilere gösterilecek spor mağazası indirimi ise `IndirimSporMagaza` sınıfındadır; her tür kullanıcıya gösterilebileceği için `reklamUygunMu` hiçbir alana bakmadan doğrudan `true` döndürür. Bu, zincirin anlamlı biçimde sonlanmasını sağlar: zincirdeki diğer reklamların hiçbiri uygun olmasa bile kullanıcıya bir reklam gösterilir ve reklam ajansı ücretini alır.

13.5 Kampanyaların kurulması ve çalıştırılması

`MainApp` içinde üç kullanıcı oluşturulur; ikisi erkek, biri kadın (ekrandan, 02:50:00): `users[0]` Erkek, C gelir düzeyi, 23 yaş; `users[1]` Erkek, A, 53 yaş; `users[2]` Kadın, B, 33 yaş.

> **Ders sonrası not:** Altyazıda kullanıcı yaşları "23, 73, 33" diye geçiyor (02:42:11); program çıktısına göre ikinci kullanıcının yaşı 53'tür (ekrandan, 02:50:00). Hoca da sonraki anlatımda 53 yaş dedi.

Zincire eklenme sırası önemlidir. Spor mağazası indirimi her kullanıcıya uygun olduğu için zincirin başına konursa geri kalan reklamlar hiç gösterilmez; diğer reklamlara hiç fırsat vermeden önlerine geçer. Hoca bunun zincirin doğru yapılandırılması gerektiğine bir başka örnek olduğunu söyledi. Bir reklam ajansı bazı reklamlarını bazı zamanlarda öne çıkarmak isteyebilir ve bunun için doğru bir yapılandırma olabilir, ama o zaman diğer reklamlar gösterilemez ve diğer müşterilerden para kazanılamaz. Bu yüzden önce sonraki reklamı olmayan `IndirimSporMagaza` oluşturulur, sonra sonrakisi o olan `FitnessSalonu`, en son da `FitnessSalonu` ile ilişkilendirilen `TrasBicak`. Üç kullanıcı da zincirin ilk elemanına verilir (ekrandan, 02:50:00):

```
public void kampanya1( ) {
    Reklam[] reklamlar = new Reklam[3];
    reklamlar[2] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2]);
    reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}

public void kampanya2( ) {
    Reklam[] reklamlar = new Reklam[4];
    reklamlar[3] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[2] = new FitnessSalonu(new Banner(), reklamlar[3]);
    reklamlar[1] = new TrasBicak(new Banner(), reklamlar[2]);
    reklamlar[0] = new OtoDergi(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    ...
}
```

Hoca beklentisini çalıştırmadan önce açıkladı: ilk kullanıcı 23 yaşında bir erkek olduğu için ona tıraş bıçağı reklamı gelmelidir (gelir düzeyi önemli değildir). İkinci kullanıcı 53 yaşında

olduğu için tıraş bıçağı reklamına uymaz; sonraki reklam 18-39 yaş arası kadınlar içindir ama kullanıcı kadın değildir; bu yüzden spor mağazası indirimi gösterilmelidir. Üçüncü kullanıcı kadın ve yaşı aralıkta olduğu için fitness salonu reklamını görmelidir. Kampanya 1'in çıktısı (ekrandan, 02:50:00):

```
dp.chainOfResponsibility.example.TrasBicak->Erkek,C,23
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->dp.chainOfResponsibility.example.IndirimSporMa
dp.chainOfResponsibility.example.IndirimSporMagaza->Erkek,A,53
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->Kadın,B,33
```

İlk kullanıcıya tıraş bıçağı reklamı gösterildi. İkinci kullanıcı için önce tıraş bıçağı reklamı bakıp kendini uygun görmedi ve isteği fitness salonuna yönlendirdi; fitness salonu da kullanıcı kadın olmadığı için uygun görmedi ve spor mağazası indirimine yönlendirdi; o da her şeye "evet" dediği için reklamını gösterdi. Üçüncü kullanıcı için tıraş bıçağı kendini uygun görmedi, fitness salonuna iletti, fitness salonu kullanıcıyı uygun görüp gösterdi. Mekanizma doğru çalıştı.

Kampanya 2'de zincirin en başına `OtoDergi` eklenir; gerisi aynıdır, yalnız dizi indisleri kayar. Hoca bunun dizi kullanmanın dezavantajı olduğunu, kopyaya çok yakın (duplicate) kod oluştuğunu söyledi. Reklamlar dizi yerine bir bağlı liste (`LinkedList`) olarak üye alanda tutulsaydı, Kampanya 2 Kampanya 1'in döndürdüğü listeyi alıp başına yalnız otomobil dergisi reklamını ekleyebilir, böylece kampanyanın yeniden kullanımı kodda da gösterilmiş olurdu.

Kampanya 2'de kullanıcılardan hiçbiri 18-29 yaş arası A gelir grubu erkek koşulunu sağlamaz: ilk kullanıcı 23 yaşında ama C gelir düzeyindedir. Hoca bu yüzden otomobil dergisi reklamının hiç gösterilmemesini bekledi ve çalıştırınca öyle oldu. Ardından ilk kullanıcının gelir düzeyini A yaptı; bu kez zincirin en başındaki otomobil dergisi reklamı kullanıcıyı hemen yakaladı ve gösterdi. Diğer kullanıcılar için çıktı Kampanya 1 ile aynı olduğu için üzerinden geçilmedi. Çıktılar python ile kurulan bir benzetimle de doğrulandı.

Hoca, derste Chain of Responsibility'ye kadar gelindiğini söyleyerek dersi kapattı.

Örnek Sorular

Soru 1: Çoklu kalıtımla çözümün zayıf yönleri (ekrandan, 00:10:00)

`ScrollBorderTextArea` sınıfını hem `ScrollTextArea`'dan hem `BorderTextArea`'dan kalıtan çözümün zayıf yönleri nelerdir?

Derste verilen cevap (slayttan): Çoklu kalıtım her dilde desteklenmez (Java ve C#'ta yoktur). Bu şekil "diamond problem" sorununun bir örneğidir: `TextArea`'daki soyut bir metot iki ara sınıfta farklı gövdelerle tanımlanmışsa en alttaki sınıfa hangisinin geçeceği belirsizdir. `TextArea`'ya ek olarak `Panel` sınıfına da kaydırma ve çerçeve yeteneği kazandırmak için üç sınıf daha gerekir ve bu durumda kaydırma ve çerçeve yeteneklerinin tek yerde gerçeklenmesi avantajı da ortadan kalkar.

Soru 2: Birden fazla üstbilgi ve altbilgi (ekrandan, 00:30:00)

Makbuz örneğinde aynı makbuzda birden fazla farklı üstbilgi ve/veya altbilgi basılması isteniyorsa (ör. `HEADER1, HEADER2, Receipt, FOOTER2` ya da `HEADER1, Receipt, FOOTER1, FOOTER2`) hangi tasarım gereksinimi karşılar?

Derste verilen cevap: Header ve footer türleri için ayrı kalıtım hiyerarşileri (Strategy) her makbuza yalnız bir header ve bir footer bağlar; her birleşim için ayrı sınıf yazmak kopyala-yapıştır kod demektir. Bu durumda gereksinimi ancak Decorator kalıbı karşılar: `SalesTicket` istenen sayıda ve türde `Header`/`Footer` dekoratörüyle sarılır.

Soru 3: Zincirleme sırası ve çıktı (derste sözlü, 00:44:23)

`SalesTicket` önce `Header2`, sonra `Header1`, sonra `Footer1` ile süslenip `prtTicket()` çağrılırsa ne basılır? En üstte `HEADER2` basılması için sıra nasıl olmalıdır?

Derste verilen cevap: Çıktı `HEADER1`, `HEADER2`, makbuz, `FOOTER1` olur. Header önce kendi işini yapıp sonra isteği ilişkili bileşene ilettiği için zincirin dışındaki (en son eklenen) header önce basılır. En üstte `HEADER2` isteniyorsa bileşen önce `Header1`, sonra `Header2` ile süslenmelidir. `Footer1`'in header'lardan önce ya da sonra eklenmesi sonucu değiştirmez. (Python benzetimiyle doğrulandı.)

Soru 4: Her bilette farklı header sırası (derste sözlü, 01:01:42)

Bir bilette `Header1`, başka bir bilette `Header2` yukarıda olsun isteniyor. Bunun için her seferinde kodda zinciri değiştirmek mi, yoksa karmaşık `switch-case` yapıları mı gerekir?

Derste verilen cevap: Hayır. Kullanıcıya bir açılır liste ya da menü sunulur ve listenin her öğesine uygun biçimde zincirlenmiş bileşenin en üst ucu bağlanır; seçilen öğenin zinciri kullanılır.

Soru 5: Flyweight ne zaman kullanılamaz? (derste sözlü, 01:26:29)

Flyweight kalıbının uygulanabilmesi için hangi koşullar gerekir? Nesne kimliği neden önemlidir?

Derste verilen cevap: Uygulama çok sayıda nesne kullanmalı ve saklama masrafı yüksek olmalı; durum bilgisinin çoğu (en azından bir kısmı) dışsallaştırılabilmeli; dışsal durum ayrılınca az sayıda paylaşılan nesne yetmeli; uygulama nesne kimliğine bağımlı olmamalıdır. Flyweight'te nesnelerin kimliği yoktur, yalnız genel bir anahtarı vardır (ör. `A` karakteri); tekil ziyaretçi sayımı gibi kimliğin gerektiği bir durumda kalıp kullanılamaz.

Soru 6: Karışımların tarifi (ekrandan, 01:41:15)

Bölüm 8.4'teki `MainApp` (Eclipse sürümü) çalıştırılırsa konsolda ne yazar?

Derste verilen cevap (program çıktısı):

```
Kara barut
75.0gr Sodyum Nitrat
15.0gr Karbon
10.0gr Kükürt
Temiz barut
75.0gr Potasyum Nitrat
20.0gr Karbon
25.0gr Kükürt
```

Gramlar `double` olduğu için `75.0` biçiminde yazılır; madde adları `MaddeFabrikasi`'ndaki paylaşılan `KimyasalMadde` nesnelerinden gelir.

Soru 7: Reklam zincirinin çıktısı (ekrandan, 02:50:00)

Kampanya 1 zinciri `TrasBicak → FitnessSalonu → IndirimSporMagaza` biçiminde kurulmuştur. Kullanıcılar (Erkek, C, 23), (Erkek, A, 53) ve (Kadın, B, 33) sırayla zincirin başına verilirse hangi reklamlar gösterilir? Kampanya 2'de zincirin başına `OtoDergi` (18-29 yaş arası A gelir grubu erkek) eklenirse ne değişir?

Derste verilen cevap: Kampanya 1'de ilk kullanıcıya tıraş bıçağı; ikinciye (tıraş bıçağı ve fitness salonundan geçerek) spor mağazası indirimi; üçüncüye (tıraş bıçağından geçerek) fitness salonu reklamı gösterilir (çıktı Bölüm 13.5'te). Kampanya 2'de hiçbir kullanıcı otomobil dergisi koşulunu sağlamadığı için her istek `OtoDergi`'den `TrasBicak`'a iletilir ve sonuçlar aynı kalır. İlk kullanıcının gelir düzeyi A yapılırsa otomobil dergisi reklamı ona gösterilir. (Python benzetimiyle doğrulandı.)

Soru 8: İndirim reklamı zincirin başına konursa (derste sözlü, 02:42:50)

`IndirimSporMagaza` zincirin başına konursa ne olur?

Derste verilen cevap: Her kullanıcıya uygun olduğu için bütün istekleri o karşılar; zincirin geri kalanındaki reklamlar hiç gösterilmez. Bu, zincirin doğru yapılandırılması gerektiğinin

bir örneğidir; herkese uygun halka zincirin sonuna konarak hem diğer reklamlara fırsat verilir hem de zincirin anlamlı sonlanması sağlanır.

Dersin Toparlaması

Bu derste yapısal kalıplardan Decorator, Facade, Flyweight ve Proxy işlendi, ardından davranışsal kalıplara geçilerek Chain of Responsibility anlatıldı. Decorator, kalıtımın derleme anında ve sınıf düzeyinde kalan katılığına karşı, nesneleri bir zincir hâlinde birbirine bağlayarak tek tek nesnelere çalışma anında sorumluluk ekler; makbuz örneği, header ve footer dekoratörlerinin farklı birleşimlerini ve zincirleme sırasının çıktıya etkisini gösterdi. Facade karmaşık bir alt sisteme tek bir erişim noktası verir. Flyweight, durum bilgisini içsel ve dışsal diye ayırarak çok sayıda benzer nesneyi paylaşılan az sayıda nesneyle temsil eder; kimya fabrikası örneğinde "değişeni bul ve sarmala" ilkesiyle gram bilgisi ayrıldı ve maddeler bir fabrikadan simgeyle paylaşıldı. Proxy, gerçek nesneyle aynı arayüzü gerçekleyen bir vekil koyarak geç yükleme, uzak erişim ya da ödeme denetimi gibi işleri istemciye fark ettirmeden ekler.

Chain of Responsibility, isteği karşılayabilecek birden fazla nesnenin bulunduğu ve hangisinin karşılayacağının önceden bilinmediği durumlarda isteği zincir boyunca iletir. Decorator'dan farkı, zincirde yalnız bir halkanın yanıt vermesidir. Reklam örneği, zincir mekanizmasının `final` bir metotla üst sınıfa konmasını ve zincirin doğru sıralanıp anlamlı biçimde sonlandırılmasının önemini gösterdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.